

```

<!-- creador.html-->
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Avatar-IA - Creador de libretos</title>
  <style>
    body { font-family: 'Segoe UI', sans-serif; background: #f4f6f9; margin: 0;
padding: 0; }
    .navbar { background: #2c3e50; padding: 15px 30px; display: flex; gap:
20px; }
    .navbar a { color: white; text-decoration: none; font-weight: bold;
font-size: 15px; opacity: 0.8; }
    .navbar a:hover, .navbar a.active { opacity: 1; border-bottom: 2px solid
#00ff00; }
    .nav-visor { margin-left: auto; color: #00ff00 !important; }
    .container { max-width: 1300px; margin: 30px auto; background: white;
padding: 30px; border-radius: 12px; box-shadow: 0 4px 15px rgba(0,0,0,0.05); }
    h1 { margin-top: 0; color: #1a1a1a; }
    .zona-importar { background: #f8f9fc; border: 1px dashed #4e73df; padding:
15px; border-radius: 8px; margin-bottom: 20px; display: flex; align-items: center;
gap: 15px; }
    .zona-importar p { margin: 0; font-size: 14px; color: #4e73df; font-weight:
bold; }
    .btn-importar { background: #4e73df; color: white; padding: 8px 15px;
border: none; border-radius: 4px; cursor: pointer; font-weight: bold; font-size:
13px; }
    #importInput { display: none; }
    table { width: 100%; border-collapse: collapse; margin-top: 20px; }
    th, td { padding: 10px; border: 1px solid #e3e6f0; text-align: left; }
    th { background: #f8f9fc; color: #4e73df; }
    input, select { width: 100%; padding: 8px; border: 1px solid #ccc;
border-radius: 4px; box-sizing: border-box; }

    .file-picker-container { display: flex; gap: 5px; align-items: center; }
    .btn-browse { background: #e3e6f0; color: #4e73df; border: 1px solid #ccc;
padding: 8px; border-radius: 4px; cursor: pointer; font-size: 12px; font-weight:
bold; }
    .btn-browse:hover { background: #dddfeb; }

    .btn { padding: 10px 15px; border: none; border-radius: 5px; font-weight:
bold; cursor: pointer; color: white; margin-top: 15px; }
    .btn-add { background: #28a745; }
    .btn-download { background: #6c5ce7; float: right; }
    .btn-delete { background: #dc3545; padding: 5px 10px; font-size: 12px; }
  </style>
</head>
<body>

```

```

<div class="navbar">
  <a href="index.html">Control</a>
  <a href="creador.html" class="active">Creador de libretos</a>
  <a href="visor.html" target="_blank" class="nav-visor">📄 Abrir Visor
(Nueva Pestaña)</a>
</div>

```

```

<div class="container">
  <h1></h1> Diseñador de Botones de acción</h1>

  <div class="zona-importar">
    <p>¿Quieres continuar un archivo existente?</p>
    <button class="btn-importar"
onclick="document.getElementById('importInput').click()">📁 Importar
config.json</button>
    <input type="file" id="importInput" accept=".json"
onchange="importarJsonExistente(this)">
  </div>

```

```

<table id="tablaConfig">
  <thead>
    <tr>
      <th>Tipo (Color)</th>
      <th>Etiqueta del Botón (Label)</th>
      <th>Archivo de Video</th>
      <th>Audio Separado (Opcional)</th>
      <th>Tiempo (seg)</th>
      <th>Texto Subtítulo</th>
      <th>Acción</th>
    </tr>
  </thead>
  <tbody>
    <!-- Las filas se inyectan dinámicamente -->
  </tbody>
</table>

```

```

  <button class="btn btn-add" onclick="agregarFila()">➕ Añadir
Botón</button>
  <button class="btn btn-download" onclick="exportarJSON()">📄 Descargar
config.json</button>
</div>

```

```

<script>
  // Fila por defecto inicial
  window.onload = function() {
    agregarFila({ tipo: 'hablar', label: 'Saludo Inicial', video:
'hablando.mp4', audio: '', tiempo: 5, subtítulo: '¡Hola a todos! Bienvenidos.' });
  };

  function actualizarVideoSugerido(selectElement) {

```

```

        const fila = selectElement.closest('tr');
        const videoInput = fila.querySelector('.video');

        if (selectElement.value === 'hablar') videoInput.value =
'hablando.mp4';
        else if (selectElement.value === 'pensar') videoInput.value =
'pensando.mp4';
        else if (selectElement.value === 'esperar') videoInput.value =
'esperando.mp4';
        else if (selectElement.value === 'oscurecer') videoInput.value = '';
    }

    // Corregido: Ahora lee el índice [0] de la lista de archivos de forma
segura
    function capturarNombreArchivo(inputElement, claseDestino) {
        const fila = inputElement.closest('tr');
        const campoTexto = fila.querySelector('.' + claseDestino);
        if (inputElement.files && inputElement.files.length > 0) {
            campoTexto.value = inputElement.files[0].name;
        }
    }

    function agregarFila(datosFila = null) {
        const tbody = document.querySelector('#tablaConfig tbody');
        const nuevaFila = document.createElement('tr');

        const tipo = datosFila ? datosFila.tipo : 'hablar';
        const label = datosFila ? datosFila.label : 'Nueva Acción';
        const video = datosFila ? datosFila.video : 'hablando.mp4';
        const audio = datosFila ? datosFila.audio : '';
        const tiempo = datosFila ? datosFila.tiempo : 4;
        const subtítulo = datosFila ? datosFila.subtítulo : '...';

        const uidVideo = 'file_v_' + Math.floor(Math.random() * 1000000);
        const uidAudio = 'file_a_' + Math.floor(Math.random() * 1000000);

        nuevaFila.innerHTML = `
            <td>
                <select class="tipo" onchange="actualizarVideoSugerido(this)">
                    <option value="hablar" ${tipo === 'hablar' ? 'selected' :
''}>🗣️ Hablar (Verde)</option>
                    <option value="pensar" ${tipo === 'pensar' ? 'selected' :
''}>💭 Pensar (Naranja)</option>
                    <option value="esperar" ${tipo === 'esperar' ? 'selected' :
''}>⌚ Esperar (Azul)</option>
                    <option value="oscurecer" ${tipo === 'oscurecer' ?
'selected' : ''}>👤 Oscurecer (Negro)</option>
                </select>
            </td>
            <td><input type="text" class="label" value="${label}"></td>

```

```

        <td>
            <div class="file-picker-container">
                <input type="text" class="video" value="${video}">
                <button class="btn-browse"
onclick="document.getElementById('${uidVideo}').click()">📁</button>
                <input type="file" id="${uidVideo}" accept="video/*"
style="display:none;" onchange="capturarNombreArchivo(this, 'video')">
            </div>
        </td>
        <td>
            <div class="file-picker-container">
                <input type="text" class="audio" value="${audio}"
placeholder="Opcional">
                <button class="btn-browse"
onclick="document.getElementById('${uidAudio}').click()">📁</button>
                <input type="file" id="${uidAudio}" accept="audio/*"
style="display:none;" onchange="capturarNombreArchivo(this, 'audio')">
            </div>
        </td>
        <td><input type="number" class="tiempo" value="${tiempo}" min="0"
step="0.1"></td>
        <td><input type="text" class="subtitulo" value="${subtitulo}"></td>
        <td><button class="btn btn-delete"
onclick="eliminarFila(this)">✖</button></td>
    `;
    tbody.appendChild(nuevaFila);
}

function eliminarFila(btn) {
    btn.closest('tr').remove();
}

function importarJsonExistente(input) {
    const archivo = input.files;
    if (!archivo || archivo.length === 0) return;
    const lector = new FileReader();
    lector.onload = function(evento) {
        try {
            const datos = JSON.parse(evento.target.result);
            document.querySelector('#tablaConfig tbody').innerHTML = '';
            datos.forEach(item => agregarFila(item));
        } catch (error) { alert("Archivo JSON no válido."); }
    };
    lector.readAsText(archivo[0]);
}

function exportarJSON() {
    const filas = document.querySelectorAll('#tablaConfig tbody tr');
    const dataConfig = [];
    filas.forEach(fila => {

```

```

        dataConfig.push({
            tipo: fila.querySelector('.tipo').value,
            label: fila.querySelector('.label').value,
            video: fila.querySelector('.video').value,
            audio: fila.querySelector('.audio').value.trim(),
            tiempo: parseFloat(fila.querySelector('.tiempo').value) || 0,
            subtítulo: fila.querySelector('.subtítulo').value
        });
    });
    const dataStr = "data:text/json;charset=utf-8," +
encodeURIComponent(JSON.stringify(dataConfig, null, 2));
    const downloadAnchor = document.createElement('a');
    downloadAnchor.setAttribute("href", dataStr);
    downloadAnchor.setAttribute("download", "config.json");
    document.body.appendChild(downloadAnchor);
    downloadAnchor.click();
    downloadAnchor.remove();
}
</script>
</body>
</html>

<!-- index.html -->
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Avatar-IA - Control</title>
    <style>
        body { font-family: 'Segoe UI', sans-serif; background: #f4f6f9; margin: 0;
padding: 0; }

        .navbar { background: #2c3e50; padding: 15px 30px; display: flex; gap:
20px; }
        .navbar a { color: white; text-decoration: none; font-weight: bold;
font-size: 15px; opacity: 0.8; }
        .navbar a:hover, .navbar a.active { opacity: 1; border-bottom: 2px solid
#00ff00; }
        .nav-visor { margin-left: auto; color: #00ff00 !important; }

        .container { max-width: 900px; margin: 30px auto; background: white;
padding: 30px; border-radius: 12px; box-shadow: 0 4px 15px rgba(0,0,0,0.05); }
        h1 { text-align: center; color: #1a1a1a; margin-bottom: 25px; }

        .zona-carga { border: 2px dashed #b2bec3; padding: 30px; text-align:
center; border-radius: 8px; background: #fafbfc; margin-bottom: 20px; }
        .btn-cargar { background: #6c5ce7; color: white; padding: 10px 20px;
border: none; border-radius: 5px; font-weight: bold; cursor: pointer; font-size:
15px; }
        #fileInput { display: none; }

```

```

        .grilla-botones { display: grid; grid-template-columns: repeat(auto-fill,
minmax(200px, 1fr)); gap: 15px; margin-top: 20px; }
        .card-boton { display: flex; flex-direction: column; background: #fafbfc;
border: 1px solid #e3e6f0; padding: 12px; border-radius: 8px; transition: all 0.3s
ease; }

        .card-boton button { width: 100%; padding: 12px; border: none;
border-radius: 6px; font-weight: bold; cursor: pointer; color: white; font-size:
14px; transition: all 0.2s; }

        /* Colores de los estados originales */
        .btn-hablar { background: #28a745; }      .btn-hablar:hover { background:
#218838; }
        .btn-pensar { background: #fd7e14; }      .btn-pensar:hover { background:
#e0670d; }
        .btn-esperar { background: #007bff; }      .btn-esperar:hover { background:
#0069d9; }
        .btn-oscurecer { background: #212529; } .btn-oscurecer:hover { background:
#111215; }

        /* ESTADO USADO: Convierte la tarjeta y el botón a gris */
        .card-boton.usado { background: #e9ecef; border-color: #ced4da; opacity:
0.7; }
        .card-boton.usado button { background: #6c757d !important; color: #f8f9fa;
}

        .info-meta { font-size: 11px; color: #858796; margin-top: 8px; text-align:
center; font-style: italic; }
    </style>
</head>
<body>

    <div class="navbar">
        <a href="index.html" class="active">Control</a>
        <a href="creador.html">Creador de libretos</a>
        <a href="visor.html" target="_blank" class="nav-visor">📖 Abrir Visor
(Nueva Pestaña)</a>
    </div>

    <div class="container">
        <h1>Avatar-IA - Control</h1>

        <div id="zonaCarga" class="zona-carga">
            <button class="btn-cargar"
onclick="document.getElementById('fileInput').click()">📁 Cargar
config.json</button>
            <input type="file" id="fileInput" accept=".json"
onchange="cargarJsonManual(this)">
        </div>

```

```

        <div id="contenedorPrincipal" class="grilla-botones"></div>
    </div>

<script>
    const canal = new BroadcastChannel('anima_ai_channel');
    let temporizadorEstado = null;
    let datosActuales = []; // Guarda la estructura del JSON activo
    let botonesUsados = []; // Guarda los índices o IDs de los botones clicados

    window.onload = function() {
        // Comprobar si existen datos previos en localStorage al refrescar
        if (localStorage.getItem('anima_json_datos')) {
            const continuar = confirm("Se detectó una sesión previa. ¿Deseas CONTINUAR donde te quedaste?\n\n(Aceptar = Continuar / Cancelar = Iniciar sesión nueva)");

            if (continuar) {
                datosActuales =
                JSON.parse(localStorage.getItem('anima_json_datos'));
                botonesUsados =
                JSON.parse(localStorage.getItem('anima_botones_usados')) || [];

                // Dibujar botones y ocultar zona de carga automáticamente
                document.getElementById('contenedorPrincipal').innerHTML = '';
                datosActuales.forEach((item, index) => crearBotonHTML(item,
index));

                document.getElementById('zonaCarga').style.display = 'none';
            } else {
                // Si elige sesión nueva, borramos el rastro antiguo
                localStorage.clear();
            }
        }
    };

    function cargarJsonManual(input) {
        // CORREGIDO: Extraemos el primer archivo de la lista [0]
        const archivo = input.files[0];
        if (!archivo) return;

        const lector = new FileReader();
        lector.onload = function(evento) {
            try {
                datosActuales = JSON.parse(evento.target.result);
                botonesUsados = []; // Reiniciamos historial para un JSON nuevo

                // Guardar base en localStorage
                localStorage.setItem('anima_json_datos',
                JSON.stringify(datosActuales));
                localStorage.setItem('anima_botones_usados',
                JSON.stringify(botonesUsados));
            } catch (error) {
                console.error('Error al cargar el JSON manual:', error);
            }
        };
        lector.readAsText(archivo);
    }

```

```

        document.getElementById('contenedorPrincipal').innerHTML = '';
        datosActuales.forEach((item, index) => crearBotonHTML(item,
index));
        document.getElementById('zonaCarga').style.display = 'none';
    } catch (error) { alert("Archivo JSON inválido."); }
};
lector.readAsText(archivo);
}

function crearBotonHTML(item, index) {
    const contenedor = document.getElementById('contenedorPrincipal');
    const card = document.createElement('div');

    // Comprobar si este botón específico ya había sido usado
    const esUsado = botonesUsados.includes(index);
    card.className = `card-boton ${esUsado ? 'usado' : ''}`;
    card.id = `card_${index}`;

    let colorClase = `btn-${item.tipo}`;
    let metaTexto = item.tiempo > 0 ? `⌚ ${item.tiempo}s` : 'Acción Directa';

    // Si ya fue usado, le añadimos el tilde visual al texto del botón
    let etiquetaTexto = esUsado ? `✓ ${item.label}` : item.label;

    const audioSeguro = item.audio ? item.audio.replace(/"/g, '&quot;') : '';

    card.innerHTML = `
        <button id="btn_${index}" class="${colorClase}"
onclick="ejecutarAccion('${item.tipo}', '${item.video}', '${audioSeguro}',
${item.tiempo}, \`${item.subtitulo || ''}\`, ${index})">
            ${etiquetaTexto}
        </button>
        <div class="info-meta">${metaTexto}</div>
    `;
    contenedor.appendChild(card);
}

function ejecutarAccion(tipo, videoArchivo, audioArchivo, tiempo,
textoSubtitulo, index) {
    if (temporizadorEstado) clearTimeout(temporizadorEstado);

    // Registrar el botón como usado (excepto comandos globales como oscurecer)
    if (tipo !== 'oscurecer' && !botonesUsados.includes(index)) {
        botonesUsados.push(index);
        localStorage.setItem('anima_botones_usados',
JSON.stringify(botonesUsados));

        // Cambiar aspecto visual al instante
        const tarjeta = document.getElementById(`card_${index}`);

```



```

        const boton = document.getElementById(`btn_${index}`);
        if (tarjeta && boton) {
            tarjeta.classList.add('usado');
            if (!boton.innerText.startsWith('✓')) {
                boton.innerText = `✓ ${boton.innerText}`;
            }
        }
    }

    if (tipo === 'oscurecer') { canal.postMessage({ comando: 'OSCURECER' });
return; }
    if (tipo === 'esperar') { canal.postMessage({ comando: 'CAMBIAR_ESTADO',
video: videoArchivo, audio: '', subtítulo: '' }); return; }

    canal.postMessage({ comando: 'CAMBIAR_ESTADO', video: videoArchivo, audio:
audioArchivo, subtítulo: textoSubtítulo, tiempo: tiempo });

    temporizadorEstado = setTimeout(() => {
        canal.postMessage({ comando: 'CAMBIAR_ESTADO', video: 'esperando.mp4',
audio: '', subtítulo: '' });
    }, tiempo * 1000);
}
</script>
</body>
</html>

<!-- visor.html -->
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Avatar-IA - Display</title>
    <style>
        html, body { margin: 0; padding: 0; width: 100%; height: 100%; background:
#000; overflow: hidden; font-family: 'Courier New', Courier, monospace; }
        video { position: absolute; top: 0; left: 0; width: 100%; height: 100%;
object-fit: cover; opacity: 0; transition: opacity 0.15s ease-in-out; z-index: 1; }
        video.activo { opacity: 1; z-index: 2; }
        .contenedor-subtítulo { position: fixed; bottom: 8%; left: 50%; transform:
translateX(-50%); background: rgba(0, 0, 0, 0.75); color: #ffffff; padding: 12px
30px; border-radius: 6px; font-size: 26px; font-family: Arial, sans-serif;
font-weight: 500; text-align: center; max-width: 80%; display: none; z-index: 15;
box-shadow: 0 4px 10px rgba(0,0,0,0.5); border: 1px solid rgba(255,255,255,0.1); }
        .pantalla-dos { position: fixed; top: 0; left: 0; width: 100%; height:
100%; background: black; color: #001aff; padding: 20px; box-sizing: border-box;
font-size: 20px; display: block; z-index: 10; }
        .cursor { animation: parpadeo 0.8s infinite; }
        @keyframes parpadeo { 0%, 100% { opacity: 1; } 50% { opacity: 0; } }
        .btn-inicio { background: #1d009b; color: rgb(255, 255, 255); border: none;
padding: 12px 24px; font-weight: bold; font-family: inherit; cursor: pointer;

```

```

margin-top: 20px; display: inline-block; font-size: 16px; }
</style>
</head>
<body>

    <div id="pantallaTerminal" class="pantalla-dos">
        <p>(C)COPYRIGHT IA CORP 2026.</p>
        <br>
        <div id="lineaPrompt">
            <!-- Separamos el texto del guion bajo usando un span con la clase .cursor -->
            <span>C:\preguntale a chatGPT></span><span class="cursor">_</span><br>
            <button class="btn-inicio" onclick="inicializarSistema()">[ EJECUTAR IA
        ]</button>
    </div>
</div>

    <div id="cajaSubtitulo" class="contenedor-subtitulo"></div>

    <video id="videoA" loop playsinline></video>
    <video id="videoB" loop playsinline></video>

    <!-- Reproductor de audio independiente invisible -->
    <audio id="reproductorAudio"></audio>

    <script>
        const vidA = document.getElementById('videoA');
        const vidB = document.getElementById('videoB');
        const audioSeparado = document.getElementById('reproductorAudio');
        const pantallaTerminal = document.getElementById('pantallaTerminal');
        const cajaSubtitulo = document.getElementById('cajaSubtitulo');

        const canal = new BroadcastChannel('anima_ai_channel');
        let sistemaListo = false;
        let reproductorActual = 'A';
        let temporizadorSubtitulo = null;

        function inicializarSistema() {
            sistemaListo = true;
            pantallaTerminal.style.display = 'none';
            vidA.src = "esperando.mp4";
            vidA.load();
            vidA.play().then(() => vidA.classList.add('activo'));
        }

        function obtenerNombreArchivo(url) {
            return url.substring(url.lastIndexOf('/') + 1);
        }

        canal.onmessage = function(evento) {
            if (!sistemaListo) return;

```

```

const data = evento.data;

if (data.comando === 'CAMBIAR_ESTADO') {
    pantallaTerminal.style.display = 'none';
    if (temporizadorSubtitulo) clearTimeout(temporizadorSubtitulo);

    const activo = (reproductorActual === 'A') ? vidA : vidB;
    const fondo = (reproductorActual === 'A') ? vidB : vidA;

    // 1. Manejo del Subtítulo (Desaparece 0.4 segundos antes de
expirar)
    if (data.subtitulo && data.subtitulo.trim() !== "") {
        cajaSubtitulo.innerHTML = data.subtitulo;
        cajaSubtitulo.style.display = 'block';
        const tiempoOcultado = (data.tiempo * 1000) - 400;
        temporizadorSubtitulo = setTimeout(() => {
cajaSubtitulo.style.display = 'none'; }, tiempoOcultado > 0 ? tiempoOcultado : 10);
    } else {
        cajaSubtitulo.style.display = 'none';
    }

    // 2. Manejo Dinámico del Audio Externo
    if (data.audio && data.audio.trim() !== "") {
        // Muteamos los reproductores de video para usar el audio
externo exclusivamente
        vidA.muted = true;
        vidB.muted = true;
        audioSeparado.src = data.audio;
        audioSeparado.load();
        audioSeparado.play().catch(e => console.log("Bloqueo de
audio"));
    } else {
        // Si no se definió audio, liberamos el sonido nativo de los
videos
        vidA.muted = false;
        vidB.muted = false;
        audioSeparado.pause();
        audioSeparado.src = "";
    }

    if (obtenerNombreArchivo(activo.src) === data.video) return;

    // 3. Intercambio de Video sin Parpadeo Negro
    fondo.src = data.video;
    fondo.load();
    fondo.onloadeddata = function() {
        fondo.onloadeddata = null;
        fondo.play().then(() => {
            fondo.classList.add('activo');
            activo.classList.remove('activo');

```

```

        activo.pause();
        reproductorActual = (reproductorActual === 'A') ? 'B' :
'A';
    }).catch(err => console.log(err));
    };
}

else if (data.comando === 'OSCURECER') {
    if (temporizadorSubtitulo) clearTimeout(temporizadorSubtitulo);
    vidA.pause(); vidB.pause(); audioSeparado.pause();
    vidA.classList.remove('activo'); vidB.classList.remove('activo');
    cajaSubtitulo.style.display = 'none';
    pantallaTerminal.style.display = 'block';
}
};
</script>
</body>
</html>

```